

Mini-Project #1: Audio to Image Signal Synthesis using a Time-Frequency Representation

Spencer G, Luke W, Rami L

September 22nd, 2025

1.0 Introduction

This mini-project uses a sum of sinusoids to synthesize a sound file of the song “Texas Fight.” We are taking an audio signal with a frequency of approximately 48kHz per second, truncating it by one-third, then recreating it using sinusoidal signals. Our goal was to display the signal using spectrograms and histograms.

The short-term Fourier transform is a variant of the Fourier series that has a window used to look at and describe specific parts of a signal at that time. It is described by the equation $S[m, k] = \text{STFT}\{x[n]\}[m, k] = \sum x[n]w[n - m]e^{-j2\pi(k/N)n}$. So our window is centered on “n” and each window overlaps with the next to more accurately sample our data.

We also used a power-law companding function. By raising the magnitude to a fractional power $p < 1$, it compresses the range of STFT values so that both quiet and strong spectral components are displayed. The equation used for companding was $F(x) = \text{sign}(x) * ((|x| + \epsilon)^p - \epsilon^p)$.

Color mapping was also used to represent complex numbers. The amplitude and phase can be used to describe complex numbers, using the equation $Ae^{j\phi}$. Where A represents the amplitude of the sinusoidal wave and ϕ represents the phase angle of the sinusoidal wave.

2.0 Loading an audio signal

In this part of the experiment, we used MATLAB and found an audio whose length was approximately 24 seconds long - in our case, we chose an audio file of our school’s fight song “Texas Fight”, which was about 28 seconds long and whose original frequency was 48kHz/s. We then converted it to resample it to 16kHz and converted the audio to mono from stereo. Finally, we truncated the audio to 448,000 samples (which is 28 seconds).

```
%Mini Project 1
%%2.0
wavFile = 'FightSong.wav';
[song, fs] = audioread(wavFile);
```

```

%play back song to test
%soundsc(song, fs);
%resample to 16kHz
newFs = 16000;
newSong = resample(song, newFs, fs);
%convert the song to mono
TexasFight0 = mean(newSong, 2);
%truncate signal to 28 seconds
sampleLength = 448000;
TexasFight = TexasFight0(1:sampleLength);
% soundsc(TexasFight, newFs);

```

2.1 Computing the STFT and inverse STFT

In this section, we first divided the audio signal into 1000 windows of 448 samples each, then used MATLAB's `fft` and `tfft` functions to create/convert both the complex and real variants of the Fourier and Inverse Fourier Series frequency spectrums for our audio file to create matrices. We used the Fourier series matrix in two ways: first, in order to observe the audio in this specific form, we converted it into a vector and recorded observations during playback. From playback, we observed that one sounded subtly more “noisier/crunchier” than the other, most likely due to loss/differences in data caused by or during conversion. The second way we used the Fourier series matrix was to create two pairs of real matrices using magnitude, phase, real, and imaginary.

```

%%2.1
%make 1000 windows size 448 samples
windowSize = 448;
windowCount = 1000;
windows = reshape(TexasFight, windowSize, windowCount);
%fourier transform the windows
S = fft(windows);
%check to make sure we didn't distort audio
recovered = ifft(S);
recovered = recovered(:);
%pause(30);
%soundsc(recovered, newFs);
%convert recovered audio into two matrices
Smag = abs(S);
Sphase = angle(S);
Sreal = real(S);
Simag = imag(S);

```

2.3 Comanding

Using the data from the previous section of this project with the two pairs of real matrices of magnitude/phase and real/imaginary, we plotted the histograms of all and wrote down our observations of their shapes and ranges, as can be read below.

```

%%2.3
%Plot histograms of recoveredMag, Phase, Real, Imag
figure;%1

```

```

histogram(Smag);
title('Histogram of STFT Magnitude');
xlabel('Magnitude');
ylabel('Number of Samples');
% Histogram of recoveredMag:
% description
figure;%2
histogram(Sphase);
title('Histogram of Phase');
xlabel('Phase angle (radians)');
ylabel('Number of Signals');
% Histogram of Phase:
% description
figure;%3
histogram(Sreal);
title('Histogram of Real Portion of Signal');
xlabel('Real Portion Coefficient');
ylabel('Number of Signals');
% Histogram of Real:
% description
figure;%4
histogram(Simag);
title('Histogram of Imaginary Portion of Signal');
xlabel('Imaginary Portion Coefficient');
ylabel('Number of Signals');
% Histogram of Imag
% description
compandedMag = F(Smag);
figure;%5
histogram(compandedMag);
title('Companded Signal Magnitude');
xlabel('Magnitude');
ylabel('Number of Signals');
%Histogram of compandedMag
% description
compandedReal = F(Sreal);
figure;%6
histogram(compandedReal);
title('Companded Real Portion of Signal');
xlabel('Real Portion Coefficient');
ylabel('Number of Signals');
%Histogram of compandedReal
% description
%%%2.4
compandedImag = F(Simag);
figure;%7
imagesc(compandedMag);
colorbar;
title('Visual Representation of Companded Magnitude');

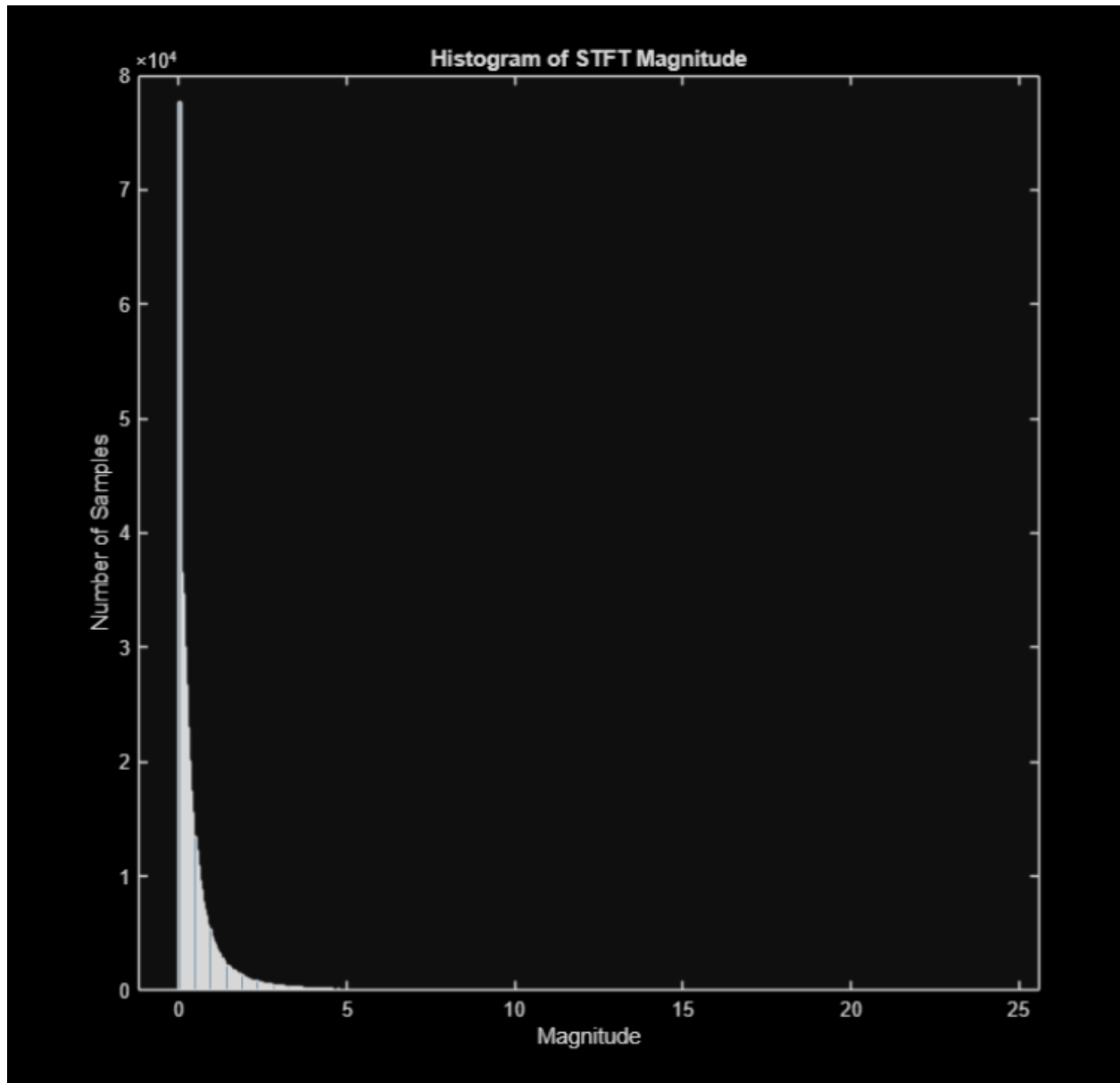
```

```

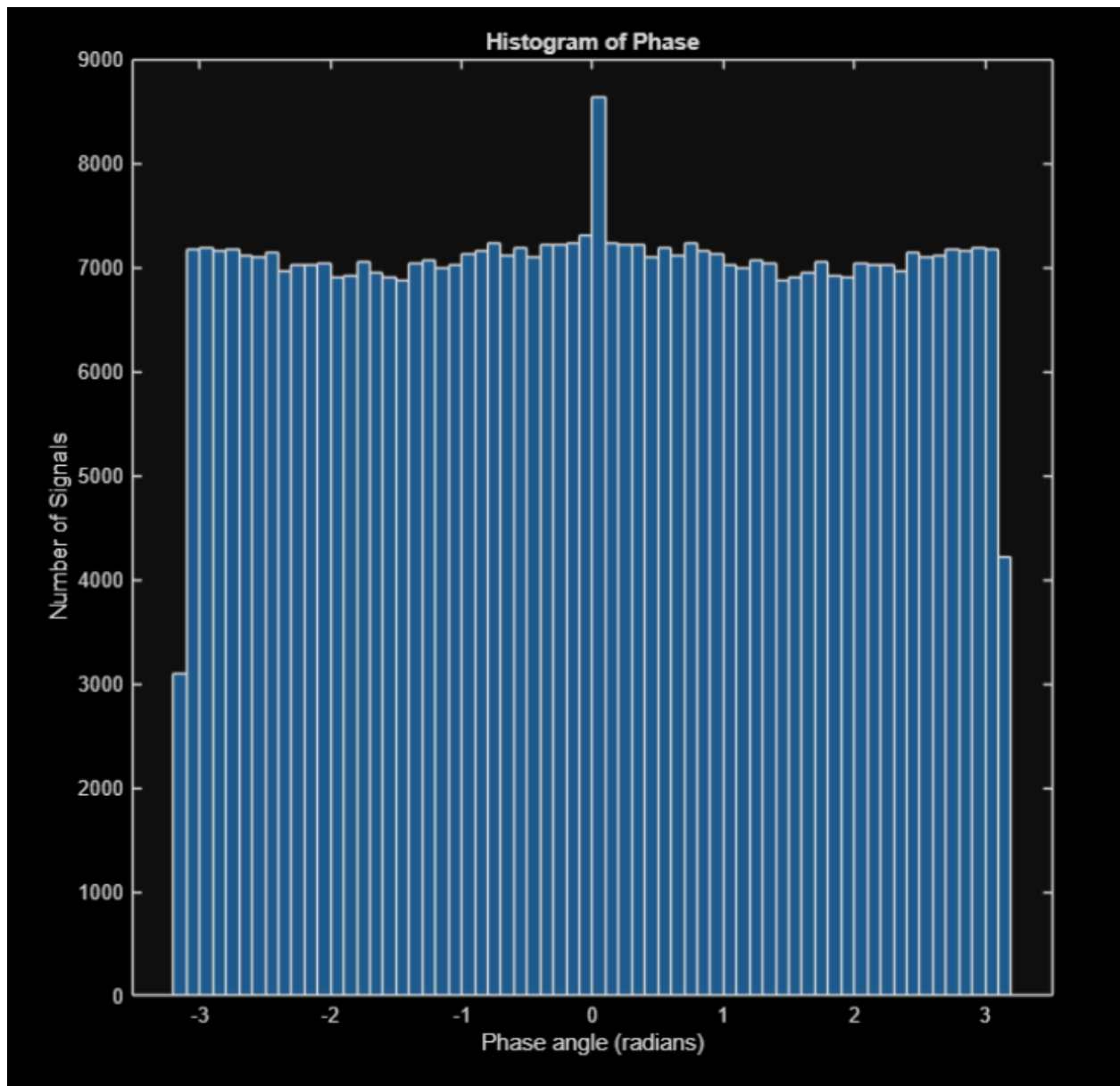
xlabel('Time?');
ylabel('Frequency?');
% description
figure;%8
imagesc(Sphase);
colorbar;
title('Visual Representation of Phase');
xlabel('Time?');
ylabel('Phase Angle?');
figure;%9
imagesc(compandedReal);
colorbar;
title('Visual Representation of Real Portion of Signal');
xlabel('Time?');
ylabel('Magnitude');
figure;%10
imagesc(compandedImag);
colorbar;
title('Visual Representation of Imaginary Portion of Signal');
xlabel('Time?');
ylabel('Magnitude');
% 2.4.2 b.)
songImage = MagPhase2RGB(compandedMag, Sphase);
imwrite(songImage, "TexasFightImage.png");
%%3.0
newRGB = imread("TexasFightImage.png");

%convert image matrix back to the audio signal vector
image2vector = RGB2Complex(newRGB);
vector2song = ifft(image2vector);
vector2song = real(vector2song(:));
%play our processed song
%pause(30);
%soundsc(real(vector2song), newFs);
%compute PSNR
PSNR = -10*log10(mean(abs(TexasFight - vector2song).^2)) + 6.02;
disp("PSNR: ");
disp(PSNR);

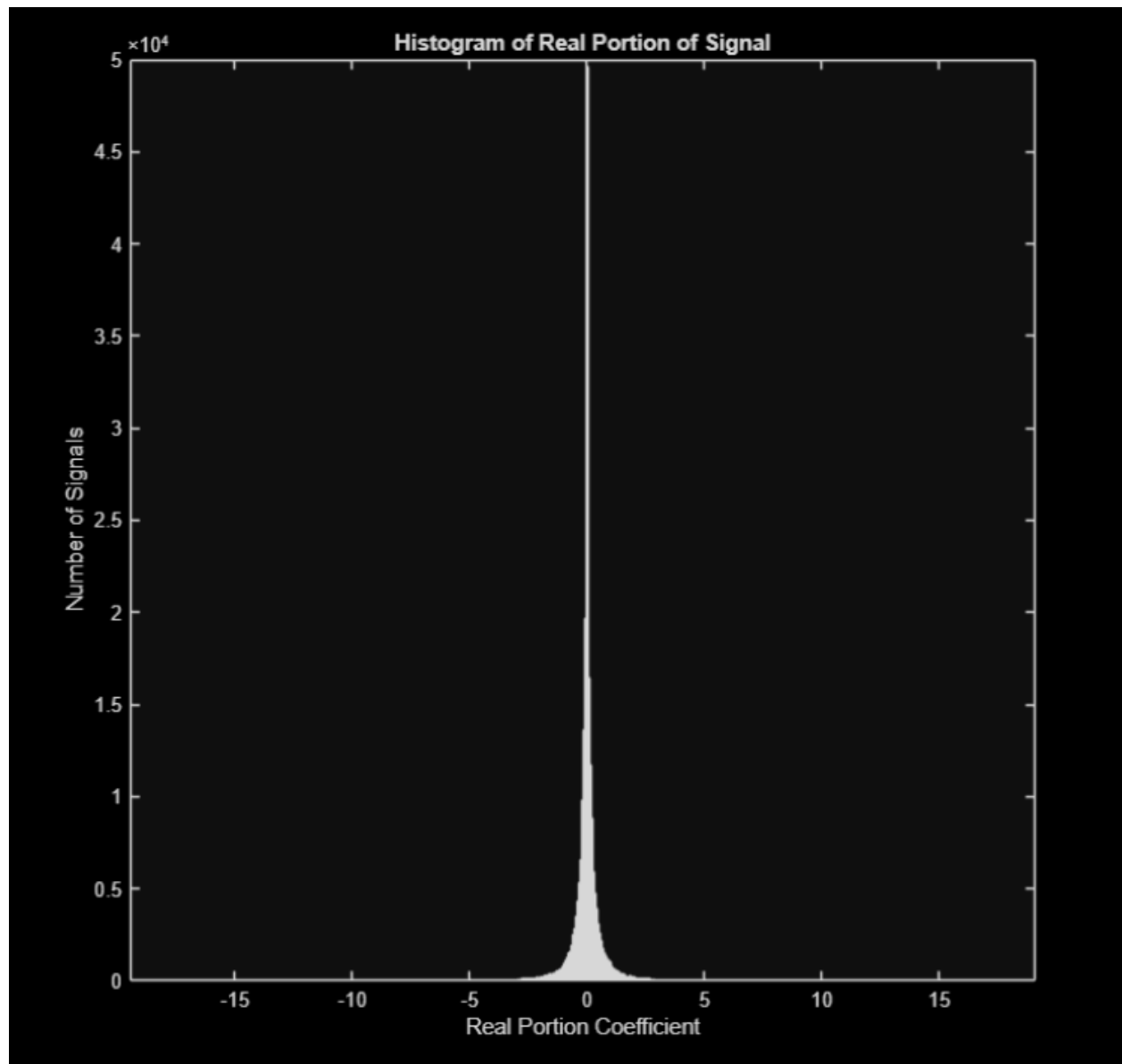
```



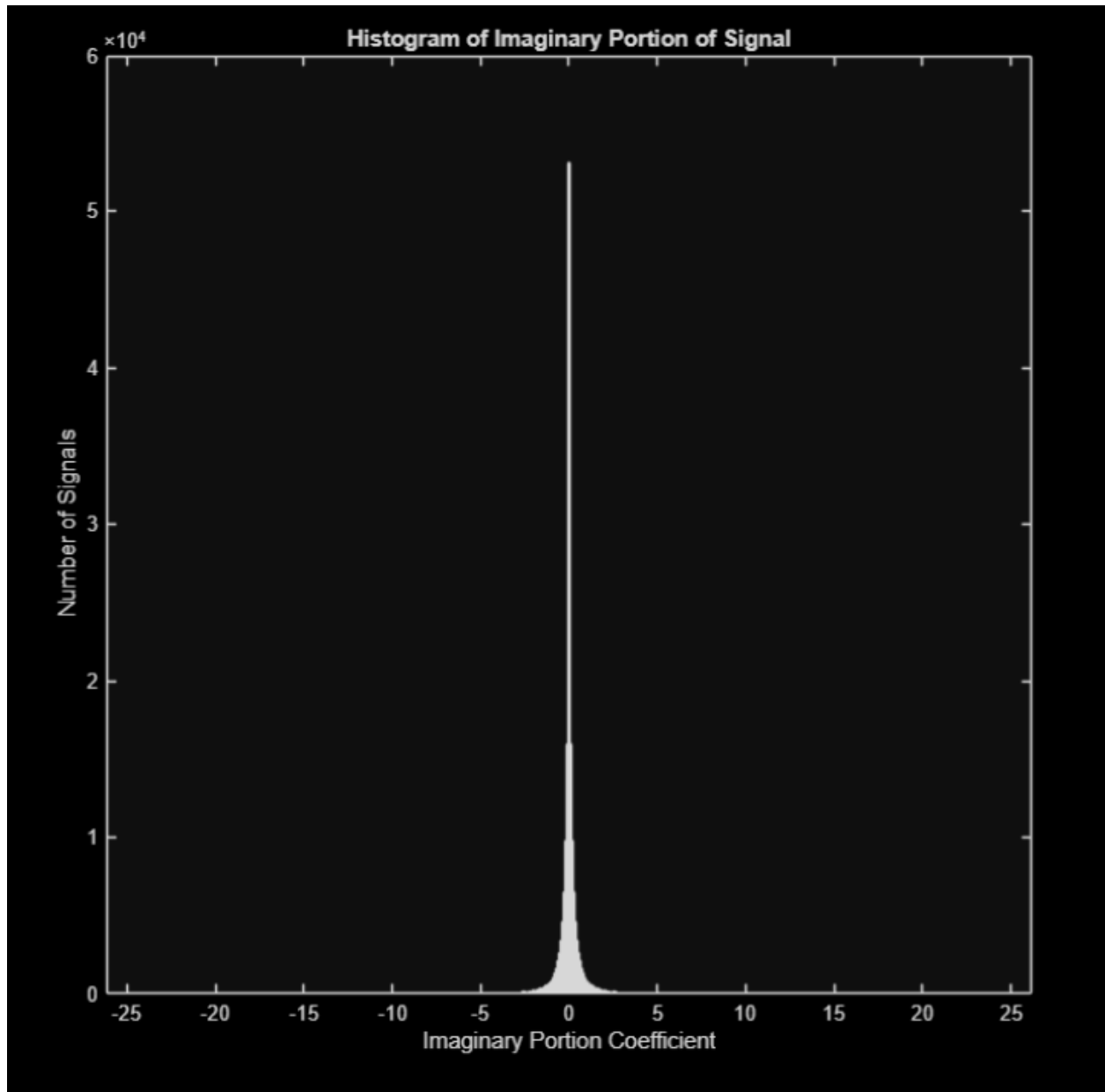
```
% Histogram of recoveredMag:  
% This graph is very one-sided with a very steep quadratic slope going from  
% a peak on the left side of the graph to leveling out on the right. The  
% graph has a range from 0 to 77644.
```



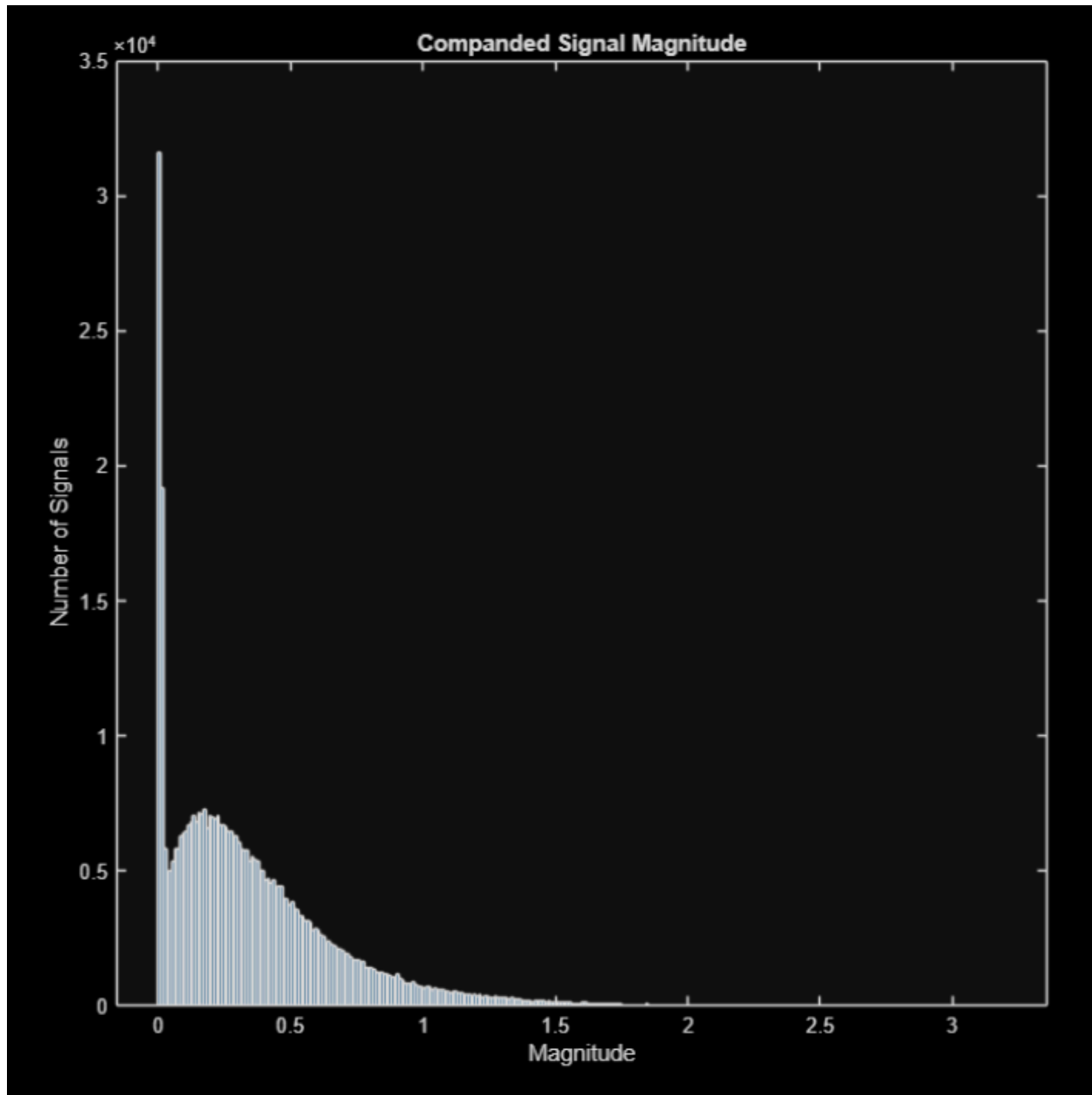
```
% Histogram of Phase:  
% The graph looks pretty uniform and has a squarish shape except for a  
% spike in the bin from [0, 0.1]. The bars on the farthest left and right  
% of the graph is also about half the size of the rest of the bars. The  
% graph has a range from 3099 to 8630.
```



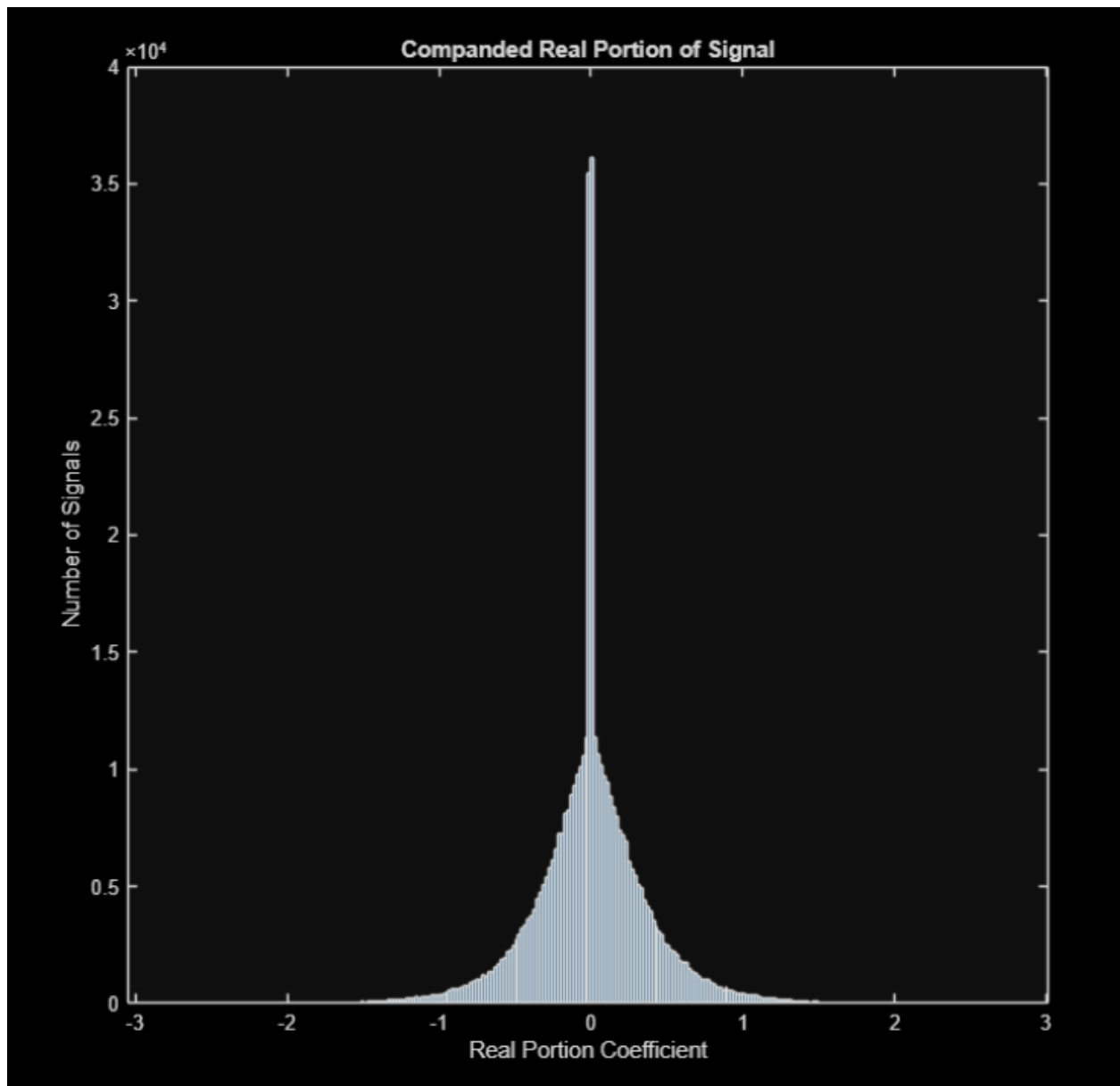
```
% Histogram of Real:  
% This graph has a very steep quadratic slope on both sides, and they meet  
% at a very tall spike in the center of the graph at x = 0. The graph has a  
% range from 0 to 49656, and once it starts increasing, it increases very  
% quickly.
```



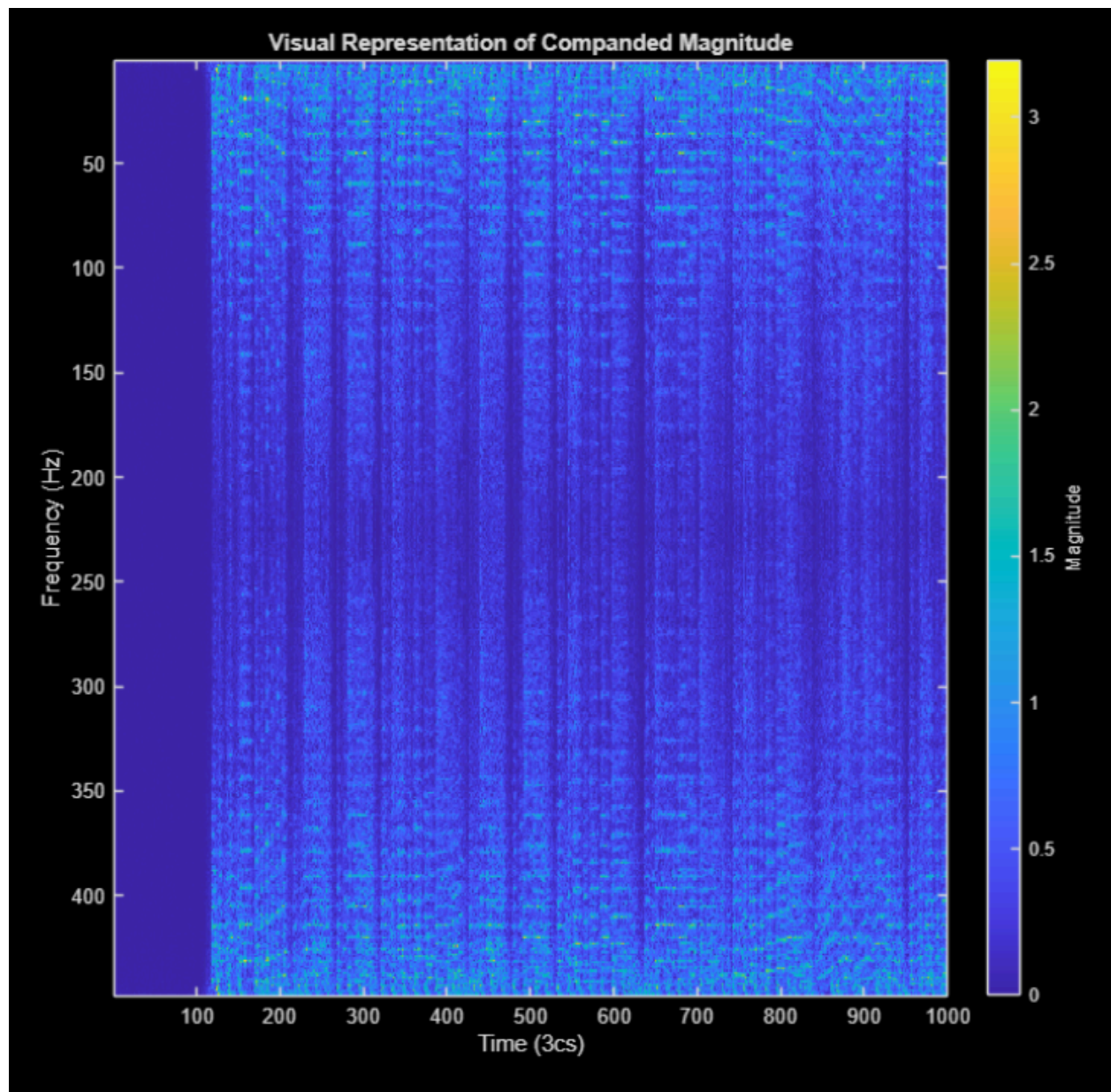
```
% Histogram of Imag:  
% This graph looks very similar to the graph of the Real Portion of the  
% Signal and it has a range from 0 to 53043. Both sides of the graph start  
% at 0 and then increase at an increasing rate to meet at a tall spike  
% centered at x = 0.
```

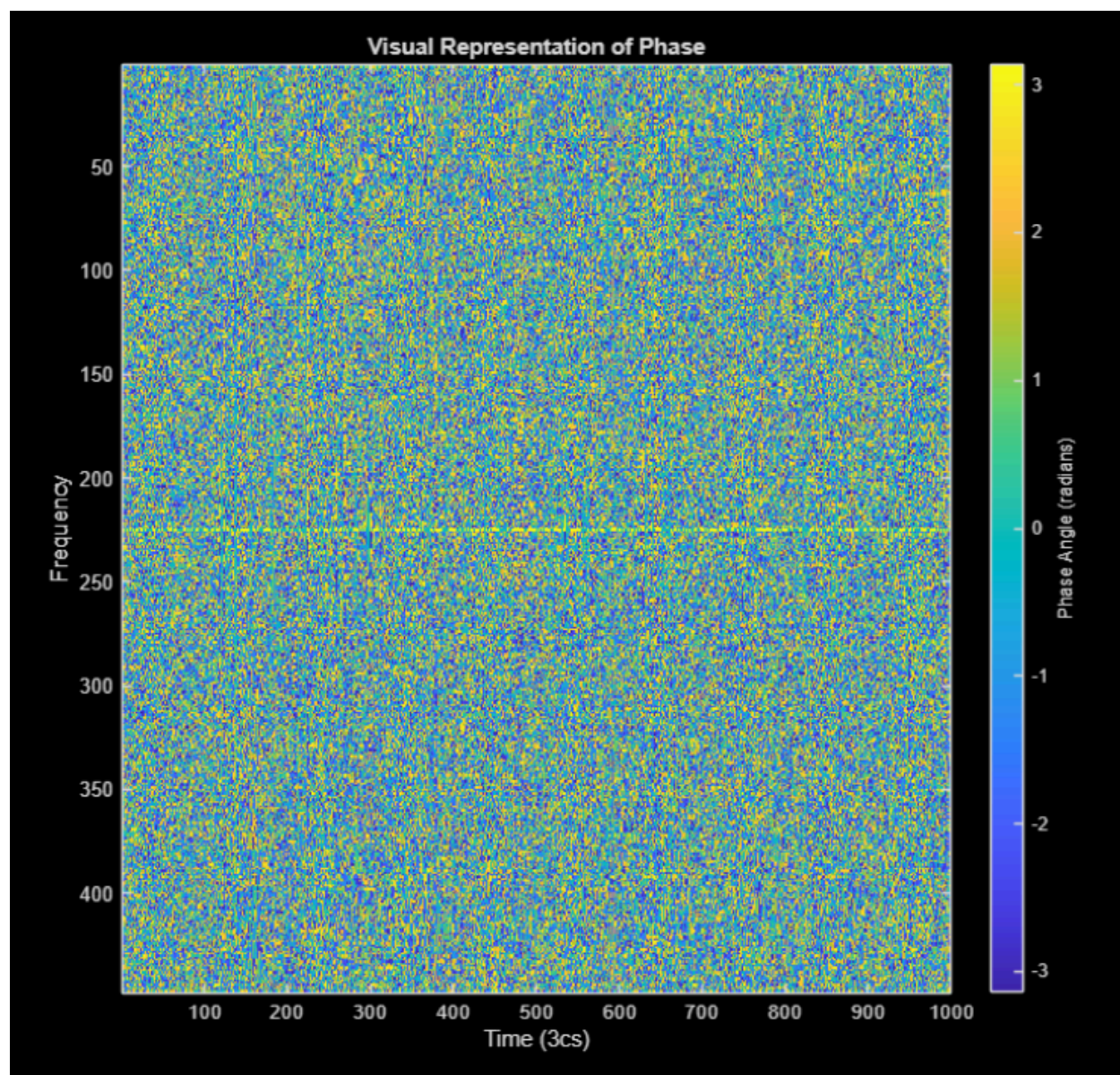
```
% Histogram of compandedMag:  
% This graph has a range from 0 to 31608. It has a very tall spike on the  
% left side of the graph, and from about x = 0.05 to x = 3, it looks like a  
% bell curve centered on x = 0.18.
```

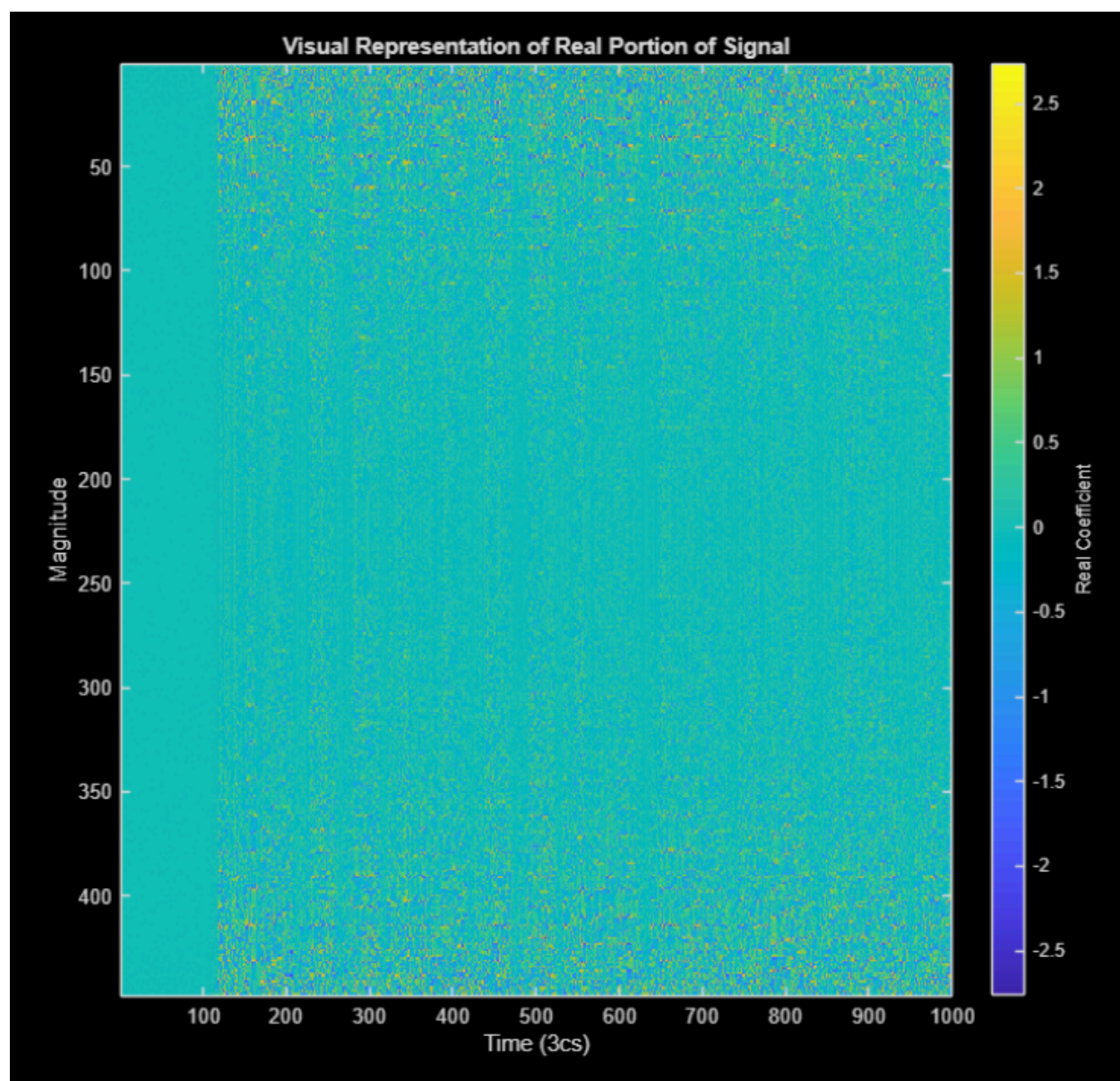


```
% Histogram of compandedReal:  
% This graph looks like a more normalized version of the graph of the real  
% portion of the signal. The slope increases at a slower rate, and the  
% peak in the middle of the graph that both sides approach is less extreme.  
% The range of this graph is from 0 to 36104.
```



% This image is very blue, and there is a solid blue block on the left of
% the image, the block represents the first couple of seconds of silence at
% the start of our sound file. You can tell where the music gets louder and
% quieter based on how blue the pixels are, blue meaning quiet, and you
% can see where the high magnitude and phase are because of a high red value
% represents a high magnitude, and a high green value represents a high
% angle.





2.4 Mapping complex values to a color image

In this section, we visualized the components of S using color mapping. Overall, you can see the sound waves in the image, but there is much more imaginary content at the upper and lower ends of the waves.

To find the intensity of the R value, we found the maximum value in our amplitude samples and divided each amplitude by that maximum. This process normalized our data to fall between 0 and 1 so that we could correlate the red intensity of a pixel directly with the amplitude of the waves in that sample. To find the intensity of the G value, we added π to each sample's phase angle, which normalized the angle between 0 and 2π . Then we divided the angles by 2π to normalize the values between 0 and 1 to accurately correlate the green intensity of a pixel with the phase angle of that sample. To find the B value, we averaged the red and green intensities and then subtracted that value from 1. The averaged values already fell between 0 and 1, so our blue intensity will also fall between 0 and 1. This makes it so that if a sample has a high phase angle and amplitude, then it will also have a high blue intensity.

```
function RGB = MagPhase2RGB(compandedMag, Sphase)
%min(compandedMag(:) = 0;
%max(compandedMag(:) ~ 3.19543189054873;
%min(Sphase(:) = -pi;
%max(Sphase(:) = pi;
R = compandedMag ./ 3.19543189054873;
G = (Sphase + pi) ./ (2*pi);
B = 1 - (R + G) ./ 2;
RGB = uint8(255*cat(3, R, G, B)); % represent as an RGB image
end
```

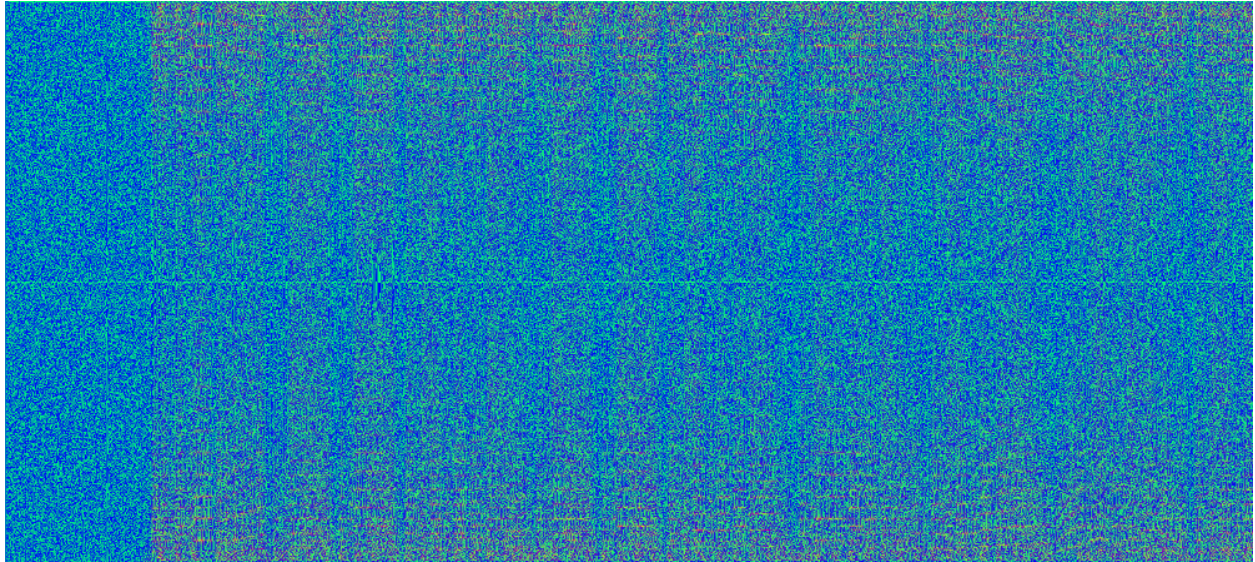
To convert our RGB values back into the amplitude and phase angle, we first divided each color's value by 255 to normalize the values between 0 and 1. Then, based on our color mapping function, we found a

matrix representation
$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ -0.5 & -0.5 \end{bmatrix} \begin{bmatrix} M \\ P \end{bmatrix}$$
 and then found the inverse of the mapping matrix,

$$\begin{bmatrix} M \\ P \end{bmatrix} = \begin{bmatrix} 0.8333 & -0.1667 & 0.3333 \\ -0.1667 & 0.8333 & 0.3333 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$
 to convert the RGB values back into the correct ranges and values to try and accurately represent our processed signal. M represents the magnitude, P represents the phase, and RGB represents their respective intensities.

```
function S1 = RGB2Complex(RGB)
R = double(RGB(:, :, 1)) ./ 255;
G = double(RGB(:, :, 2)) ./ 255;
B = double(RGB(:, :, 3)) ./ 255;
FSmag = R*0.8333 + G*-0.1667 + B*-0.3333;
Sphase = R*-0.1667 + G*0.8333 + B*-0.3333;
FSmag = FSmag.*3.19543189054873;
Sphase = (2*pi.*Sphase) - pi;
```

```
S1 = Finv(FSmag) .* exp(1j*Sphase);
End
```



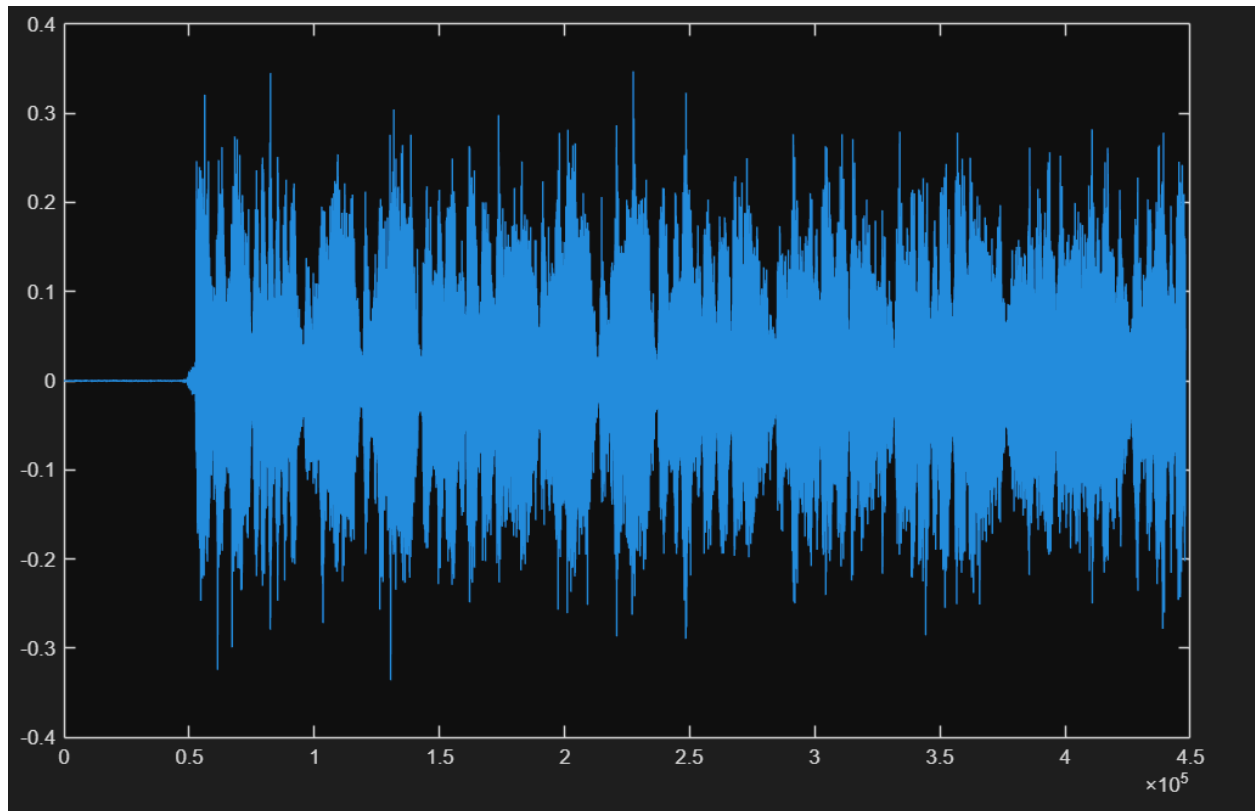
This is our final picture of the audio signal. It is a 1000x448 pixel image representing the amplitude and phase of the audio signals as RGB values in each pixel.

3.0 Recovering audio from the time-frequency image

After we used the audio signal to create the image based on the signal's amplitude and phase angle, we normalized the RGB components back into our original range and then used a decompanding function and an inverse Fourier transformation to convert the image back into a processed audio signal.

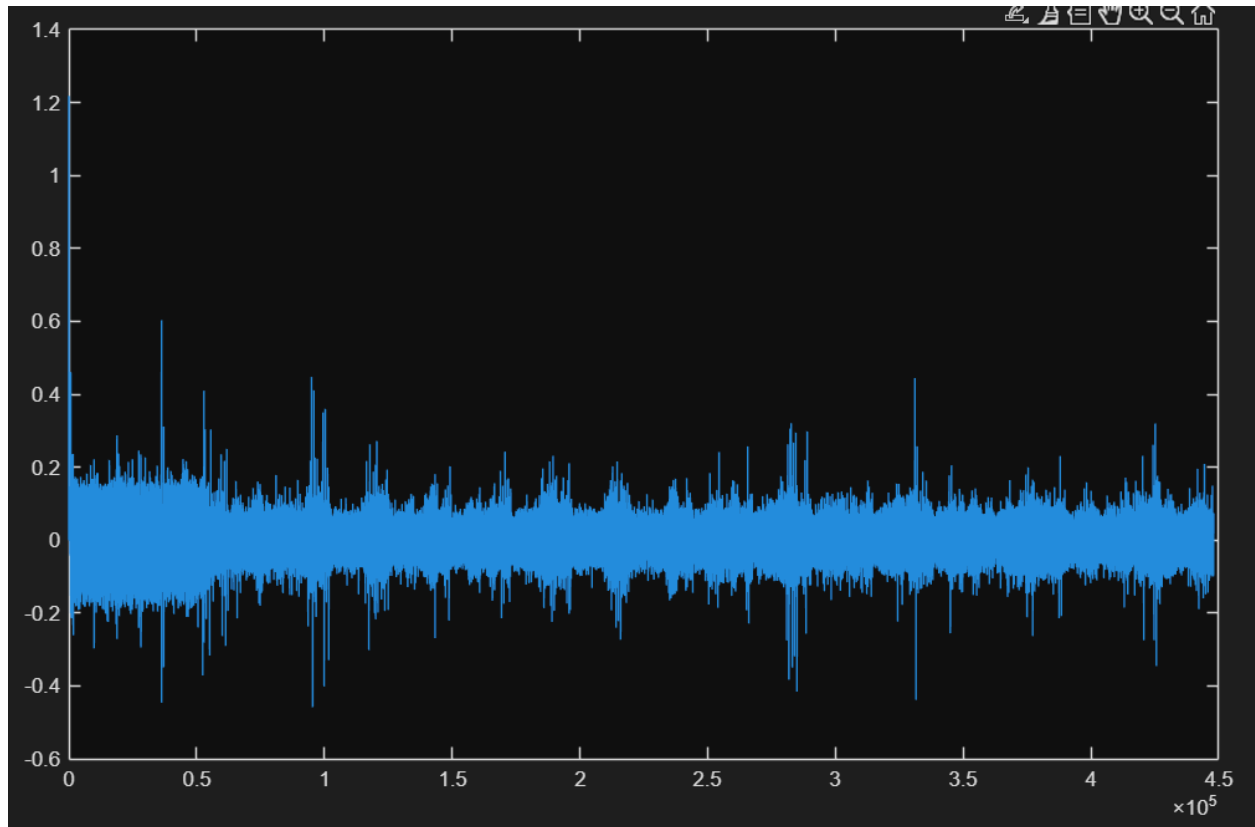
```
%convert image matrix back to the audio signal vector
image2vector = RGB2Complex(newRGB);
vector2song = ifft(image2vector);
vector2song = real(vector2song(:));
%play our processed song
pause(30);
soundsc(vector2song, newFs);
%compute PSNR
PSNR = -10*log10(mean(abs(TexasFight - vector2song).^2)) + 6.02;
disp("PSNR: ");
disp(PSNR);
% PSNR = 30.333500156401
```

Original Audio Signal



This is an Image of the original audio signal represented with amplitude on the y-axis and time(ms) on the x-axis.

Processed Audio Signal



This is an image of the processed signal represented with amplitude on the y-axis and time(ms) on the x-axis

Conclusion

The recovered song is very, very noisy, and the actual music we hear is very quiet compared to the original. The silence that used to be at the beginning sounds like white noise. The music is very distorted, but if we turn our volume up, we can still make out how the song goes. We believe the distortion made the amplitude of the noise signals feel higher than the amplitude of the signals that make up the song. The blue concentrated lines in the middle are representative of when the band takes breaths, and the sound is quieter.

The audio was so distorted because of the way that we truncated our data to fit within the range of an RGB pixel. When we sampled the data from the signal, it had precision up to 16 bits, then after the Fourier series and companding, we normalized the data to be between 0 and 1. After the normalization, we truncated the intensity of each color to have an 8-bit precision. This truncating of the data really distorted the signal and made the noise prevalent.